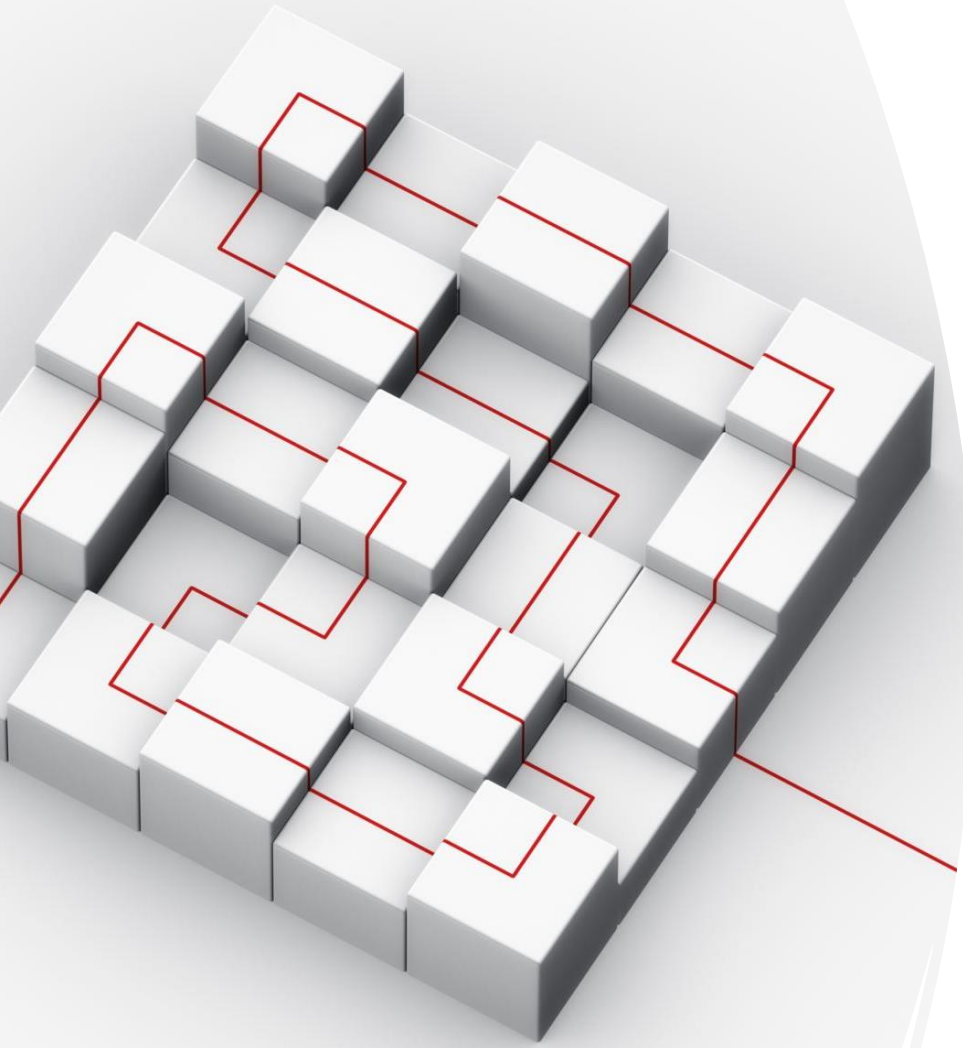
INT 161



Basic Backend Development

Week-05

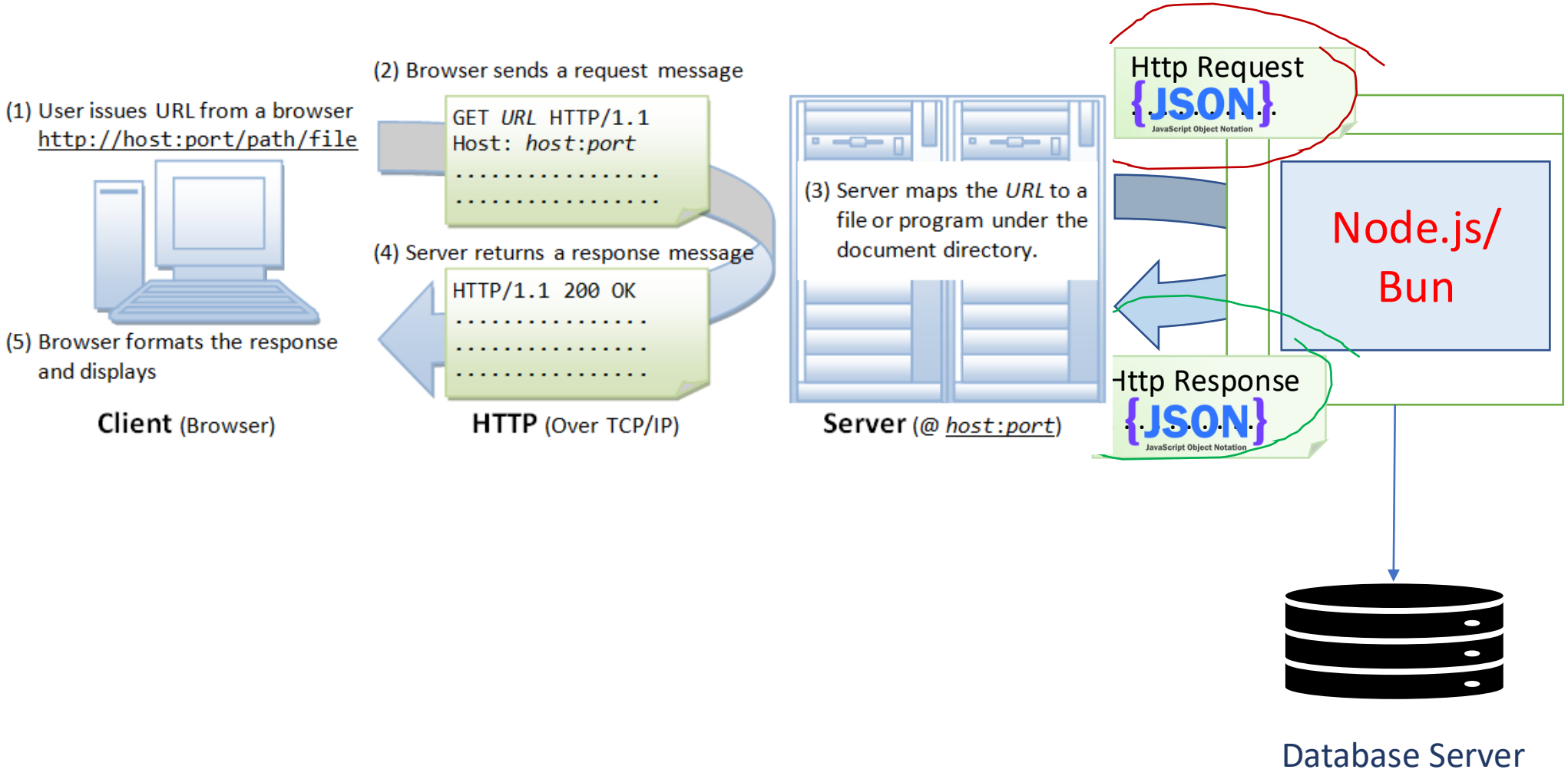
`Express.js Framework`



Unit Objectives

- After completing this unit, you should be able to:
  - Create Node.js Project with Express js framework
  - Explain basic concept of express framework
  - Explain the layer system of RESTful API principle
  - Create basic CRUD REST API follow the layer system

Web Application



What is Express JS?

- Express is the minimalist, unopinionated, fast web framework for Node.js.
- It provides a structured layer of routing and middleware execution on top of Node's built-in HTTP server.
- The Express.js framework makes it very easy to develop an application which can be used to handle multiple types of requests like the GET, PUT, POST and DELETE requests.
- Express is the backend part of something known as the MEAN/MERN stack.
  - MongoDB Express.js Angular.js Node.js
  - MongoDB Express.js React.js Node.js

Key Concepts

- Routing - Express provides a straightforward API to handle incoming HTTP requests (GET, POST, PUT, DELETE, etc.) based on URLs and paths:
- Middleware - Functions have access to the request object (req), response object (res), and the next function in the cycle. They are used for tasks like:
  - Parsing JSON request bodies (express.json())
  - Authentication and authorization
  - Logging and error handling
  - Enabling CORS (Cross-Origin Resource Sharing)
- Unopinionated Architecture
  - You have full freedom to structure your application and choose your stack.

Installation & Using Express

1. **Create Project**
2. **Install Express:**

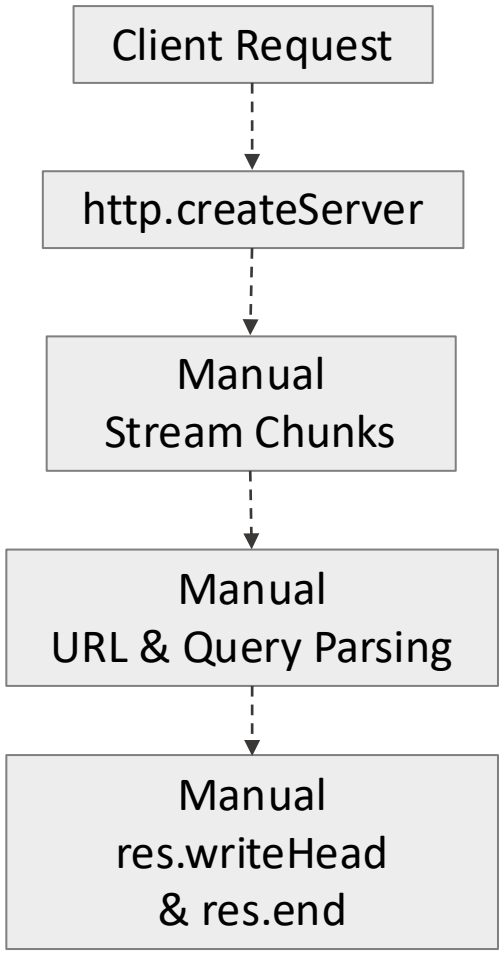
   ```
   npm install express
   ```

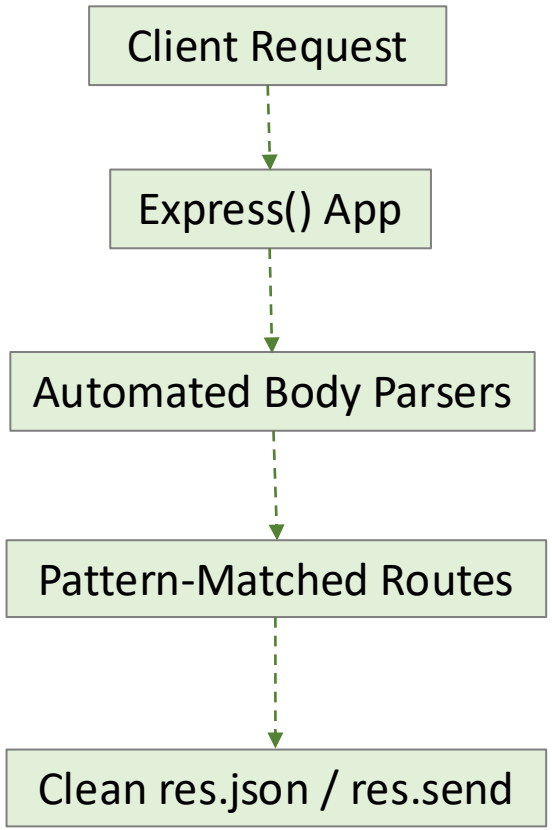
   to create folder in node_modules and install the latest stable express to project .
3. **Specify Version:** Add `@version` after the package name to install a specific Express version.
4. **Import Express:** Use

   ```
   import express from 'express';
   ```

   to include Express in your code.

Raw Node.js HTTP vs. Express.js






```
import express from 'express';

const app = express();
app.use(express.json());

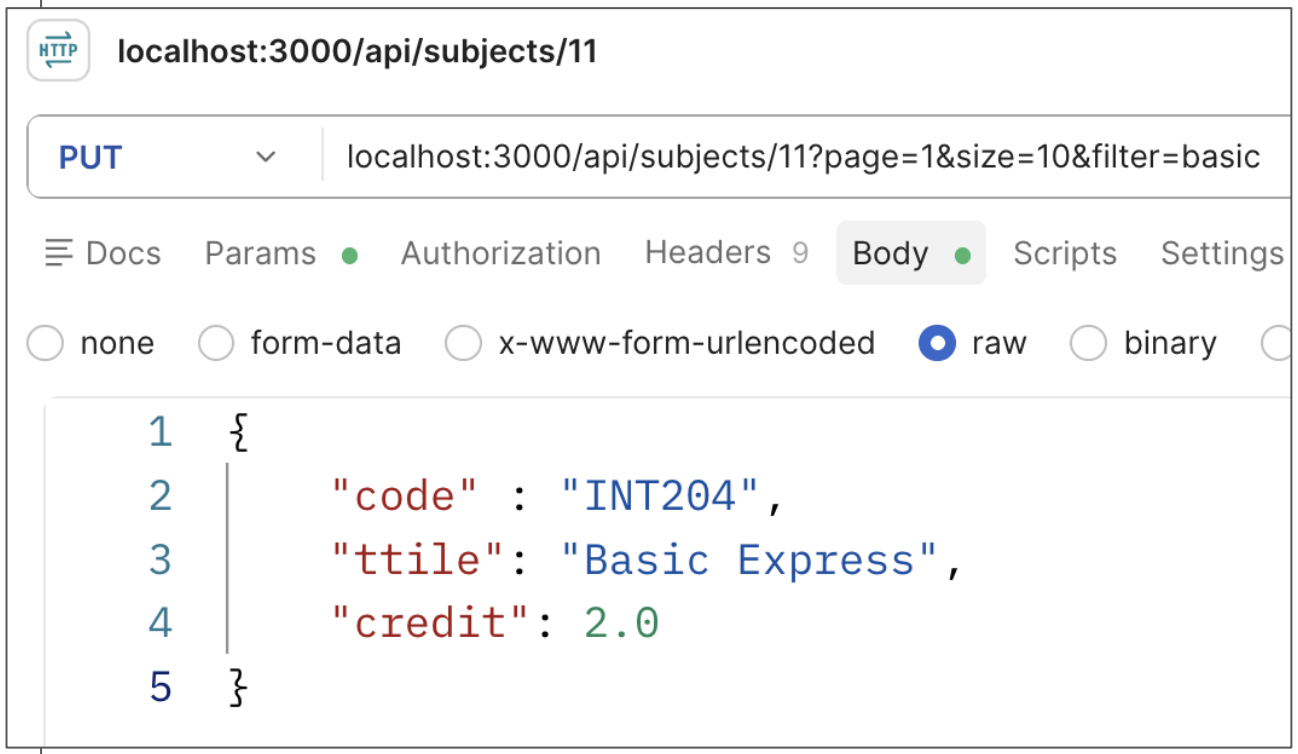
app.put('/api/subjects/:id', (req, res) => {
    const {filter, page, size} = req.query;
    const id = req.params.id;
    const subject = req.body;
    subject.id = id;
    res.json([subject, {filter: filter,
        page: page, size: size}]);
});
app.listen(3000);
```

Express.js example

```
import express from 'express';

const app = express();
app.use(express.json()); //midle-ware

app.put('/api/subjects/:id', (req, res) => {
    const {filter, page, size} = req.query;
    const id = req.params.id;
    const subject = req.body;
    subject.id = id;
    res.json([subject, {filter: filter,
        page: page, size: size}]);
});
app.listen(3000);
```



Express request parser

- Request objects provide access to various components of HTTP requests, including:
  - req.params: Extracts parameters embedded within the URL path.

```
app.put('/api/subjects/:id', (req, res) => {
   const id = req.params.id;
```

  - req.query: Retrieves parameters from the query string portion of the URL.

```
const {filter, page, size} = req.query;
```

  - req.body: Contains the parsed body of the request, typically used with middleware.

```
const {code, title, credit} = req.body;
```

```
const subject = req.body;
```

Handling Responses

`res.json()`

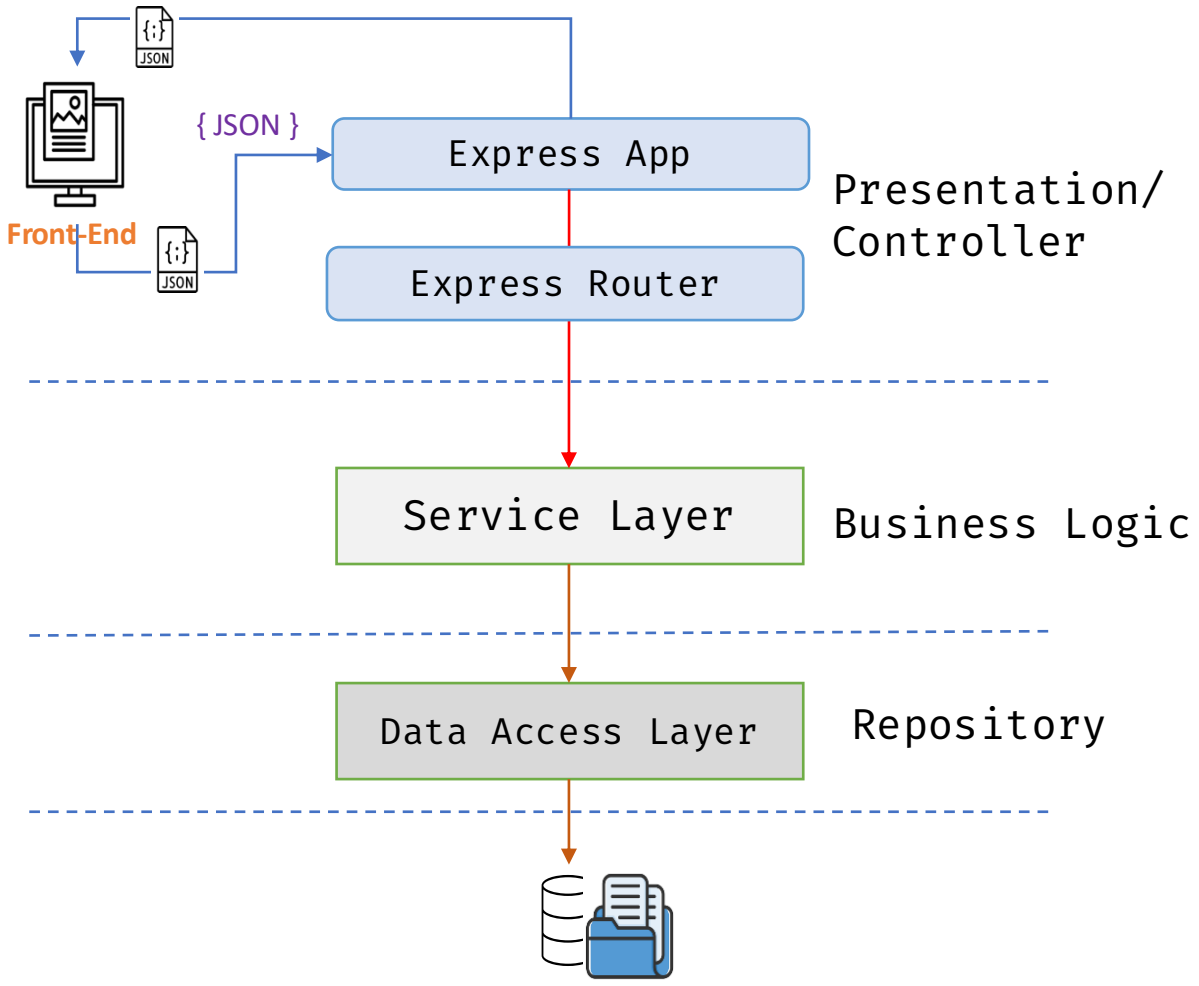
- Convert a JavaScript object or value into a JSON string for the response (status 200).

`res.status()`

- Used to set the HTTP status code for the HTTP response before sending it back to the client
- Crucially, res.status() alone does not send the response. It returns the Express res (response) object, which allows you to chain it with sending methods like .send(), .json(), or .render()

```
res.status(201).json(subject);
```

Layered System




```
import express from "express";

const PORT = 3000;
const app = express();
app.use(express.json());

app.use('/customers', customerRoute)
app.use('/orders', orderRouter);
app.use('/products', productRouter);
app.use('/account', accountRouter);

app.listen(PORT, () => {
   console.log(
   `Running at http://localhost:${PORT}`)
});
```

Layer System Functions (1/2)

- Controller (API Layer)
  - Acts as the entry point for HTTP requests (GET, POST, PUT, DELETE).
  - Converts incoming requests → objects that the service layer can process.
  - Sends back HTTP responses (status codes + JSON body).
  - Does not contain business logic, only delegates.
  - Analogy: Reception desk at a hospital — takes the patient info and directs them to the right doctor.
- Service Layer (Business Logic Layer)
  - Contains the core business logic of the application.
  - Applies rules, validations, calculations.
  - Coordinates between Controller and Repository.
  - Can call multiple repositories or external APIs to fulfill business processes.
  - Analogy: The doctor at the hospital — examines, diagnoses, and decides treatment.

Layer System Functions (2/2)

- Repository Layer (Data Access Layer)
  - Responsible for data persistence and retrieval.
  - Provides an abstraction over the database (CRUD operations).
  - No business logic — only raw data access.
  - Analogy: The hospital records office — stores and retrieves patient files.

Layer	Responsibility	Should Contain	Should Not Contain
Controller	Handle HTTP requests/responses	Routing, mapping, validation	Business logic, DB code
Service	Business rules & process coordination	Business logic, calculations	HTTP or DB details
Repository	Data access (CRUD)	Queries, persistence	Business logic, HTTP

Routing in Express

- Routing decides how an app handles client requests to a specific URL and HTTP method (GET, POST, etc.), Syntax:

```
router.METHOD(PATH, HANDLER);
```

  - METHOD can be GET, POST, PUT, DELETE, etc.
- Example:

```
router.get('/home', (req, res) => {
    res.send('Welcome to the homepage!');
});
```

- Route patterns can be strings or regex.
- Strings may include parameters, Example:

```
router.get('/api/user/:id', (req, res)
```

Router example

```
import express from "express";
import * as service from "../services/subject-service.js";
const router = express.Router();

router.get('/', async (req, res, next) => {
    const subjects = await service.getAllSubjects();
    res.json(subjects);
});
router.get('/:id', async (req, res, next) => {
    const {id} = req.params;
    const [subject] = await service.getSubjectById(id);
    if (!subject[0])
        return res.status(404).json({message: 'Subject not found'});
    res.json(subject[0]);
});
router.post('/', async (req, res, next) => {
    // call service
});
export default router;
```

Middleware in Express

- Middleware acts as an intermediary step that processes requests before they reach the router.
- For instance, it can verify user authentication prior to granting access to admin API.
- To pass control to the subsequent middleware function, use the next() method.
- Common categories of middleware include:
  - Application-level Middleware
  - Router-level Middleware
  - Error-handling Middleware
  - Built-in Middleware
  - Third-party Middleware

```
import express from 'express’;
const app = express();

app.use(express.json()); //midleware
app.use('/api/subjects, subjRouter);

app.listen(3000);
```

WebStorm Project Template - ESM

- Create project
- Install modules
- Add “type” : “module” to package.json
- Create folders
  - src
    - configs
    - repositories
    - services
    - routes
- Create Configulations file
- Create app.js/router/service/repository
- Tools -> Save project as template : Express-ESM

```
{
  "name": "basic-rest-express",
  "version": "1.0.0",
  "description": "",
  "main": "src/app.js",
  "type": "module",
      |
      |
}
```